```
"""
gmail_ingestion.py
==================
Connects to Gmail via the official Google Workspace Gmail API, pulls unread
saved-search alerts from Zillow / Redfin / Realtor.com, and parses them into
`Listing` objects.

WHY THIS MODULE IS DEFENSIVE
----------------------------
1. Alert-email HTML is a moving target. Zillow and Redfin restyle their
   templates regularly, so pure regex parsing WILL break. The design here is
   regex-first (cheap, deterministic) with an optional Claude fallback for the
   fields regex missed. You inject the fallback so this module has no hard
   dependency on the Anthropic SDK.

2. **Listing agent email is usually absent.** Saved-search alerts are marketing
   emails; they link to the listing page, they do not hand you the agent's
   inbox. Realistically most listings arrive with `agent.email = None` and the
   pipeline will generate the LOI but stop short of a draft. Options to close
   that gap are documented in the README under "Agent enrichment".

3. We do NOT scrape Zillow/Redfin/Realtor.com listing pages. Their terms of
   service prohibit automated access, and their bot defenses are effective.
   Everything here works from email you were legitimately sent.
"""

from __future__ import annotations

import base64
import logging
import re
from email.utils import parseaddr
from pathlib import Path
from typing import Any, Callable, Iterable, Optional

from bs4 import BeautifulSoup

import config
from models import Listing, ListingAgent, ListingSource, Provenance

log = logging.getLogger(__name__)

LLMExtractor = Callable[[str], dict[str, Any]]
```

```
# ---------------------------------------------------------------------------
# Authentication
# ---------------------------------------------------------------------------

def get_gmail_service(
    credentials_path: Path = config.GMAIL_CREDENTIALS_PATH,
    token_path: Path = config.GMAIL_TOKEN_PATH,
    scopes: Optional[list[str]] = None,
):
    """Build an authorised Gmail API service object.

    First run opens a browser for OAuth consent and writes `token.json`.
    Subsequent runs reuse and silently refresh that token.
    """
    # Imported here, not at module scope, so the email parser below can be
    # used in environments without the Google libraries installed.
    from google.auth.transport.requests import Request
    from google.oauth2.credentials import Credentials
    from google_auth_oauthlib.flow import InstalledAppFlow
    from googleapiclient.discovery import build

    scopes = scopes or config.GMAIL_SCOPES
    creds = None

    if token_path.exists():
        creds = Credentials.from_authorized_user_file(str(token_path), scopes)

    if not creds or not creds.valid:
        if creds and creds.expired and creds.refresh_token:
            log.info("Refreshing expired Gmail token")
            creds.refresh(Request())
        else:
            if not credentials_path.exists():
                raise FileNotFoundError(
                    f"OAuth client file missing: {credentials_path}\n"
                    "Create it in Google Cloud Console -> APIs & Services -> "
                    "Credentials -> OAuth client ID -> Desktop app."
                )
            flow = InstalledAppFlow.from_client_secrets_file(
                str(credentials_path), scopes
            )
            creds = flow.run_local_server(port=0)
        token_path.write_text(creds.to_json())
        log.info("Wrote Gmail token to %s", token_path)

    return build("gmail", "v1", credentials=creds, cache_discovery=False)
```

```
# ---------------------------------------------------------------------------
# Message retrieval
# ---------------------------------------------------------------------------

def fetch_alert_messages(
    service,
    query: str = config.GMAIL_SEARCH_QUERY,
    max_results: int = config.GMAIL_MAX_MESSAGES,
) -> list[dict[str, Any]]:
    """Return full message resources matching the Gmail search query."""
    from googleapiclient.errors import HttpError
    try:
        resp = (
            service.users()
            .messages()
            .list(userId="me", q=query, maxResults=max_results)
            .execute()
        )
    except HttpError as exc:
        log.error("Gmail list failed: %s", exc)
        return []

    ids = [m["id"] for m in resp.get("messages", [])]
    log.info("Gmail query matched %d message(s)", len(ids))

    messages: list[dict[str, Any]] = []
    for mid in ids:
        try:
            msg = (
                service.users()
                .messages()
                .get(userId="me", id=mid, format="full")
                .execute()
            )
            messages.append(msg)
        except HttpError as exc:
            log.warning("Could not fetch message %s: %s", mid, exc)
    return messages


def mark_as_read(service, message_id: str) -> None:
    """Remove the UNREAD label so the same alert is not processed twice."""
    from googleapiclient.errors import HttpError
    try:
        service.users().messages().modify(
```

```
            userId="me", id=message_id, body={"removeLabelIds": ["UNREAD"]}
        ).execute()
    except HttpError as exc:
        log.warning("Could not mark %s read: %s", message_id, exc)


# ---------------------------------------------------------------------------
# MIME walking
# ---------------------------------------------------------------------------

def _decode(data: str) -> str:
    return base64.urlsafe_b64decode(data.encode("utf-8")).decode(
        "utf-8", errors="replace"
    )


def _walk_parts(part: dict[str, Any]) -> Iterable[dict[str, Any]]:
    yield part
    for sub in part.get("parts", []) or []:
        yield from _walk_parts(sub)


def extract_message_text(message: dict[str, Any]) -> str:
    """Flatten a Gmail message into readable plain text.

    Prefers text/plain; falls back to stripping tags from text/html.
    """
    payload = message.get("payload", {})
    plain_chunks: list[str] = []
    html_chunks: list[str] = []

    for part in _walk_parts(payload):
        mime = part.get("mimeType", "")
        data = part.get("body", {}).get("data")
        if not data:
            continue
        try:
            decoded = _decode(data)
        except Exception:  # noqa: BLE001 - malformed base64 in the wild
            continue
        if mime == "text/plain":
            plain_chunks.append(decoded)
        elif mime == "text/html":
            html_chunks.append(decoded)

    if plain_chunks:
        return "\n".join(plain_chunks)
```

```
    if html_chunks:
        soup = BeautifulSoup("\n".join(html_chunks), "lxml")
        for tag in soup(["script", "style"]):
            tag.decompose()
        # Keep hrefs -- listing URLs live there.
        for a in soup.find_all("a", href=True):
            a.append(f" [{a['href']}] ")
        return soup.get_text("\n", strip=True)

    return ""


def _header(message: dict[str, Any], name: str) -> str:
    for h in message.get("payload", {}).get("headers", []):
        if h.get("name", "").lower() == name.lower():
            return h.get("value", "")
    return ""


def detect_source(message: dict[str, Any]) -> ListingSource:
    sender = _header(message, "From").lower()
    if "zillow" in sender:
        return ListingSource.ZILLOW
    if "redfin" in sender:
        return ListingSource.REDFIN
    if "realtor" in sender or "move.com" in sender:
        return ListingSource.REALTOR
    return ListingSource.UNKNOWN


# ---------------------------------------------------------------------------
# Field extraction (regex layer)
# ---------------------------------------------------------------------------

_PRICE_RE = re.compile(r"\$\s?([\d]{2,3}(?:,\d{3})+|\d{5,7})(?!\s?/\s?(?:mo|sq))"
_BEDS_RE = re.compile(r"(\d+(?:\.\d)?)\s*(?:bd|bds|beds?|bedrooms?)\b", re.I)
_BATHS_RE = re.compile(r"(\d+(?:\.\d)?)\s*(?:ba|baths?|bathrooms?)\b", re.I)
_SQFT_RE = re.compile(r"([\d,]{3,7})\s*(?:sq\.?\s?ft|sqft|square feet)\b", re.I)
_DOM_RE = re.compile(
    r"(\d+)\s*days?\s*(?:on|on the)\s*(?:market|zillow|redfin)", re.I
)
_ZIP_RE = re.compile(r"\b(97\d{3})\b")
_PHONE_RE = re.compile(r"\(?\b(\d{3})\)?[-.\s]?(\d{3})[-.\s]?(\d{4})\b")
_EMAIL_RE = re.compile(r"\b[\w.+-]+@[\w-]+\.[\w.-]+\b")
_URL_RE = re.compile(
    r"https?://(?:www\.)?(?:zillow|redfin|realtor)\.com/[^\s\]\)>\"']+", re.I
```

```
)
_ADDRESS_RE = re.compile(
    r"(\d{1,6}\s+(?:[NSEW]{1,2}\s+)?[A-Za-z0-9'.\- ]{2,40}"
    r"(?:St|Street|Ave|Avenue|Rd|Road|Dr|Drive|Ln|Lane|Ct|Court|Way|Pl|Place|"
    r"Blvd|Boulevard|Ter|Terrace|Cir|Circle|Loop|Hwy|Highway|Pkwy|Parkway)"
    r"\.?(?:\s+(?:Unit|Apt|#)\s?[\w-]+)?)",
    re.I,
)

# Vendor domains we must never mistake for a listing agent's address.
_VENDOR_EMAIL_DOMAINS = (
    "zillow.com", "redfin.com", "realtor.com", "move.com", "convey.com",
    "noreply", "no-reply", "donotreply", "mail.", "email.", "mktg.",
)


def _to_int(raw: Optional[str]) -> Optional[int]:
    if not raw:
        return None
    try:
        return int(raw.replace(",", "").replace("$", "").strip())
    except ValueError:
        return None


def _find_city(text: str) -> Optional[str]:
    """Match against the known Clackamas city list, longest name first."""
    lowered = text.lower()
    for city in sorted(config.CLACKAMAS_CITIES, key=len, reverse=True):
        if re.search(rf"\b{re.escape(city)}\b", lowered):
            return city.title()
    return None


def _find_agent_email(text: str) -> Optional[str]:
    """Return the first address that is plausibly a human agent, not a vendor."""
    for candidate in _EMAIL_RE.findall(text):
        low = candidate.lower()
        if any(bad in low for bad in _VENDOR_EMAIL_DOMAINS):
            continue
        return candidate
    return None


def _find_motivation_keywords(text: str) -> list[str]:
    low = text.lower()
    return [kw for kw in config.MOTIVATION_KEYWORDS if kw in low]
```

```
def parse_listing_from_text(
    text: str,
    *,
    source: ListingSource = ListingSource.UNKNOWN,
    message_id: Optional[str] = None,
    llm_extractor: Optional[LLMExtractor] = None,
) -> Optional[Listing]:
    """Parse one listing out of an alert email body.

    Returns None if we cannot even establish an address, which means the email
    was probably a digest, a market report, or a template we do not handle.
    """
    if not text or len(text) < 40:
        return None

    prov: dict[str, str] = {}

    addr_match = _ADDRESS_RE.search(text)
    address = addr_match.group(1).strip() if addr_match else None
    if address:
        prov["address"] = Provenance.REGEX.value

    price = _to_int(_PRICE_RE.search(text).group(1)) if _PRICE_RE.search(text) el
    beds = float(_BEDS_RE.search(text).group(1)) if _BEDS_RE.search(text) else No
    baths = float(_BATHS_RE.search(text).group(1)) if _BATHS_RE.search(text) else
    sqft = _to_int(_SQFT_RE.search(text).group(1)) if _SQFT_RE.search(text) else
    dom = int(_DOM_RE.search(text).group(1)) if _DOM_RE.search(text) else None
    zipc = _ZIP_RE.search(text).group(1) if _ZIP_RE.search(text) else None
    url_m = _URL_RE.search(text)
    city = _find_city(text)

    for key, val in (("list_price", price), ("bedrooms", beds),
                     ("bathrooms", baths), ("square_feet", sqft),
                     ("days_on_market", dom), ("zip_code", zipc),
                     ("city", city)):
        if val is not None:
            prov[key] = Provenance.REGEX.value

    # --- optional Claude fallback for whatever regex missed ---------------
    if llm_extractor and (address is None or beds is None or price is None):
        try:
            log.debug("Regex incomplete; calling LLM extractor")
            llm = llm_extractor(text[:12000])
            address = address or llm.get("address")
            city = city or llm.get("city")
```

```
            zipc = zipc or llm.get("zip_code")
            price = price if price is not None else _to_int(str(llm.get("list_pri
            beds = beds if beds is not None else llm.get("bedrooms")
            baths = baths if baths is not None else llm.get("bathrooms")
            sqft = sqft if sqft is not None else llm.get("square_feet")
            dom = dom if dom is not None else llm.get("days_on_market")
            for key in ("address", "city", "zip_code", "list_price",
                        "bedrooms", "bathrooms", "square_feet",
                        "days_on_market"):
                prov.setdefault(key, Provenance.LLM.value)
        except Exception as exc:  # noqa: BLE001
            log.warning("LLM extraction failed, continuing with regex only: %s",

    if not address:
        log.debug("No address found; skipping message %s", message_id)
        return None

    agent_email = _find_agent_email(text)
    phone_m = _PHONE_RE.search(text)
    agent = ListingAgent(
        email=agent_email,
        phone=f"({phone_m.group(1)}) {phone_m.group(2)}-{phone_m.group(3)}"
        if phone_m else None,
        source=Provenance.REGEX if agent_email else Provenance.MISSING,
    )

    listing = Listing(
        listing_id=message_id or f"{source.value}:{abs(hash(address))}",
        address=address,
        city=city or "",
        zip_code=zipc,
        list_price=price,
        bedrooms=beds,
        bathrooms=baths,
        square_feet=sqft,
        days_on_market=dom,
        motivation_keywords_found=_find_motivation_keywords(text),
        description=text[:4000],
        listing_url=url_m.group(0) if url_m else None,
        agent=agent,
        source=source,
        source_message_id=message_id,
        field_provenance=prov,
        raw_text=text[:20000],
    )

    if listing.motivation_keywords_found:
```

```
        listing.field_provenance["motivation"] = Provenance.REGEX.value

    return listing


# ---------------------------------------------------------------------------
# Top-level ingestion
# ---------------------------------------------------------------------------

def ingest(
    service=None,
    *,
    llm_extractor: Optional[LLMExtractor] = None,
    mark_read: bool = config.GMAIL_MARK_READ,
) -> list[Listing]:
    """Pull unread listing alerts and return parsed Listings (deduplicated)."""
    service = service or get_gmail_service()
    messages = fetch_alert_messages(service)

    listings: list[Listing] = []
    seen: set[str] = set()

    for msg in messages:
        text = extract_message_text(msg)
        source = detect_source(msg)
        listing = parse_listing_from_text(
            text,
            source=source,
            message_id=msg.get("id"),
            llm_extractor=llm_extractor,
        )
        if not listing:
            continue

        dedupe_key = listing.slug
        if dedupe_key in seen:
            log.debug("Duplicate listing skipped: %s", listing.address)
            continue
        seen.add(dedupe_key)
        listings.append(listing)

        if mark_read:
            mark_as_read(service, msg["id"])

    log.info("Parsed %d unique listing(s) from %d message(s)",
             len(listings), len(messages))
    return listings
```

```
def sender_address(message: dict[str, Any]) -> str:
    """Utility: RFC-5322 address of the sender."""
    return parseaddr(_header(message, "From"))[1]
```